```
  def my_attribute
    @my_attribute
  end
end

obj = MyClass.new
obj.my_attribute = 'x'
obj.my_attribute        # => "x"
```

Writing methods like these (also called *accessors*) gets boring quickly. As an alternative, you can generate accessors by using one of the methods in the Module#attr_* family. Module#attr_reader generates the reader, Module#attr_writer generates the writer, and Module#attr_accessor generates both:

```
class MyClass
  attr_accessor :my_attribute
end
```

All the attr_* methods are defined on class Module, so you can use them whenever self is a module or a class. A method such as attr_accessor is called a *Class Macro*. Class Macros look like keywords, but they're just regular class methods that are meant to be used in a class definition.

Spell: Class Macro

"Now that you know about Class Macros," Bill says, "I think I know a place in Bookworm's source code where we can make good use of them."

Class Macros Applied

The Book class in the Bookworm source code has methods named GetTitle, title2, and LEND_TO_USER. By Ruby's conventions, these methods should be named title, subtitle, and lend_to, respectively. However, there are other projects that use the Book class, and you have no control over these projects. If you just rename the methods, you will break the callers.

Bill has an idea to fix this situation: you can rename the methods if you invent a *Class Macro* (117) that deprecates the old names:

class_definitions/deprecated.rb

```
class Book
  def title # ...

  def subtitle # ...

  def lend_to(user)
    puts "Lending to #{user}"
    # ...
  end
```